Expert Viva-Voce Preparation Guide

Project: CU AI Advisor

Candidate: Manik | MCA Final Year

This guide contains 50 professionally curated questions covering Project Architecture, Technology Stack, Security, and Future Scope.

Section 1: Project Overview & Objectives

Q1: What is the primary goal of the CU AI Advisor?
Answer: The goal is to provide a 24/7 intelligent, privacy-first academic advising system that helps students with course recommendations, university policies, and appointment scheduling.

Q2: Why did you choose a 'Serverless' architecture?
Answer: To eliminate backend maintenance and compute costs while ensuring the system can scale infinitely without requiring a dedicated server for AI inference.

Q3: How does this project benefit Chandigarh University?
Answer: It reduces the administrative load on human advisors by automating routine inquiries about attendance, grading, and program details.

Q4: What is the significance of 'Privacy-First' in your project?
Answer: It means student queries are processed locally in the browser. No conversational data is sent to a central server for AI processing, ensuring 100% data sovereignty.

Q5: What was the main motivation behind this project?
Answer: The massive student growth at CU made manual advising a bottleneck; I wanted to create a modern, natural-language interface for existing institutional data.

Section 2: Technical Architecture & Design

Q6: Explain the 3-Tier Architecture of your system.
Answer: Tier 1: Presentation (Streamlit/Puter.js), Tier 2: Logic (Python/Sentiment Analysis), Tier 3: Data (SQLAlchemy/Relational DB).

Q7: What is SQLAlchemy and why did you use it?
Answer: SQLAlchemy is an Object-Relational Mapper (ORM). It allows me to interact with the database using Python objects, making the system engine-agnostic (switchable between SQLite and PostgreSQL).

Q8: Define the term 'Client-Side AI Orchestration'.
Answer: It refers to triggering AI models directly within the user's browser via WebAssembly (WASM) or SDKs like Puter.js, rather than making API calls to a centralized server.

Q9: What are the main tables in your database schema?
Answer: Users (Auth), Courses (Catalog), Policies (University Rules), InteractionLogs (History), and Appointments (Schedules).

Q10: Why is your database normalized to 3NF?
Answer: To eliminate data redundancy and ensure referential integrity, preventing anomalies during updates or deletions.

Section 3: AI & NLP Implementation

Q11: Which LLM are you using and how is it integrated?

Answer: I am using gpt-4o-mini integrated via a headless browser bridge using the Puter.js Cloud SDK.

Q12: How do you perform Sentiment Analysis?

Answer: I use the TextBlob library in Python to calculate the 'Polarity' score of user messages, ranging from -1.0 (Negative) to +1.0 (Positive).

Q13: What happens if the AI model is slow or unavailable?

Answer: The system has a 'Local Fallback' mechanism using regex-based keyword matching to answer critical university policy questions with 100% uptime.

Q14: How do you prevent 'AI Hallucinations' regarding CU policies?

Answer: By injecting specific knowledge-base data from my relational database into the AI's 'System Prompt' to bound its responses to factual CU data.

Q15: Explain the logic behind Course Recommendations.

Answer: I match student interest keywords (e.g., 'coding') against the 'interests' field in the Courses table using a weighted keyword-matching algorithm.

Section 4: Security & Authentication

Q16: How are user passwords stored in your database?

Answer: Passwords are never stored in plain text; they are hashed using the SHA-256 algorithm to ensure security.

Q17: What is Role-Based Access Control (RBAC) in your project?

Answer: It distinguishes between 'Student' users (who can chat and book) and 'Admin' users (who can access the analytics dashboard and logs).

Q18: How do you prevent SQL Injection?

Answer: By using SQLAlchemy ORM, which automatically uses parameterized queries, effectively neutralizing SQL injection risks.

Q19: Is the student's personal data safe during the AI interaction?

Answer: Yes, because of the client-side inference model; the conversational context stays in the student's browser memory.

Q20: How do you handle session management?

Answer: I use Streamlit's internal `session_state` to track authenticated users and their roles throughout a single session.

Section 5: Testing & Quality Assurance

Q21: What testing strategies did you employ?

Answer: I used a multi-level strategy: Unit Testing (Logic), Integration Testing (DB-UI), and End-to-End (E2E) Testing via Playwright.

Q22: Why did you use Playwright for E2E testing?
Answer: Playwright allows for automated simulation of real user interactions in the browser, ensuring the AI bridge works across different scenarios.

Q23: How do you test the accuracy of policy lookups?
Answer: I have a suite of test cases in `test_database.py` that verify the system retrieves the exact policy description for specific keywords.

Q24: What is 'Sentiment Polarity' tracking in your testing logs?
Answer: It's a way to measure the 'emotional tone' of user queries during stress tests to see if users are getting frustrated by the bot's responses.

Q25: Explain a critical bug you found and fixed.
Answer: During testing, I found that the AI bridge sometimes timed out; I resolved this by implementing an asynchronous 'Thinking...' state in the UI.

Section 6: Implementation Details

Q26: Why choose Streamlit over React or Angular?
Answer: Streamlit allows for rapid development of data-centric UIs using pure Python, which was ideal for integrating with my Python-based NLP engine.

Q27: How does the 'Appointment Booking' module work?
Answer: It captures user inputs (Date/Time), validates them against business rules (Mon-Fri, 9-5), and saves the transaction to the relational database.

Q28: Explain the significance of the `requirements.txt` file.
Answer: It lists all external Python dependencies (pandas, textblob, sqlalchemy) required to replicate the development environment.

Q29: How do you handle database migrations?
Answer: The `init_db()` function in `database.py` ensures the schema is automatically created or verified every time the application starts.

Q30: What is the role of `puter_bridge` in your code?
Answer: It acts as the 'Headless Driver' that connects the Streamlit backend to the browser's JavaScript environment to execute AI tasks.

Section 7: Results & Future Scope

Q31: What are the key findings of your project?
Answer: 1. Serverless AI is technically feasible for institutions. 2. Client-side inference dramatically reduces hosting costs. 3. Privacy-first design increases student trust.

Q32: How can this system be integrated with CU-IMS?
Answer: By replacing the local SQLite file with a connection string to the CU-IMS central PostgreSQL/Oracle database via SQLAlchemy.

Q33: What is the future scope for multi-lingual support?

Answer: I plan to integrate translation APIs or multi-lingual LLMs to support regional languages like Hindi and Punjabi.

Q34: Can this chatbot handle voice commands?

Answer: In the next version, I intend to use the Web Speech API to allow students to speak their queries instead of typing.

Q35: How would you scale this for 50,000 students?

Answer: The serverless architecture is already scalable; I would only need to move the database to a cloud-managed service like AWS RDS.

Section 8: Miscellaneous / Logic

Q36: What is a 'Knowledge Base' in the context of your bot?

Answer: It's the collection of verified university data (Courses/Policies) stored in our database that the bot uses to provide accurate answers.

Q37: Define 'Natural Language Processing' (NLP).

Answer: NLP is the field of AI that enables computers to understand, interpret, and generate human language, which we use via the LLM.

Q38: How do you ensure the bot stays 'Professional'?

Answer: Through 'System Prompt Engineering'—I explicitly instruct the AI to behave as a CU Academic Advisor in its core instructions.

Q39: What is the 'Admin Dashboard' for?

Answer: It's a management tool for university staff to see interaction trends, average student sentiment, and upcoming appointments.

Q40: How do you handle 'Negative Sentiment'?

Answer: The admin dashboard flags interactions with low sentiment scores, allowing staff to identify and resolve student grievances proactively.

Section 9: Core Concepts

Q41: What is a 'Relational Database'?

Answer: A database structured to recognize relations between stored items of information, using tables with rows and columns.

Q42: What is 'Asynchronous Programming'?

Answer: A method of programming that allows a task to run separately from the main application thread, which we use for AI inference calls.

Q43: Define 'WASM' (WebAssembly).

Answer: A binary instruction format that allows high-performance code (like AI models) to run in web browsers at near-native speed.

Q44: What is 'Sentiment Polarity' vs 'Subjectivity'?

Answer: Polarity is the emotional tone (positive/negative), while Subjectivity is the measure of how much opinion vs. fact is in the text.

Q45: Explain the 'Salted Hashing' concept.

Answer: Adding a unique, random string (salt) to a password before hashing it to protect against pre-computed rainbow table attacks.

Section 10: Conclusion Prep

Q46: What was the biggest technical challenge you faced?

Answer: Bridging the Python backend with the browser-based AI SDK (Puter.js) while maintaining a seamless user experience.

Q47: If you had more time, what one feature would you add?

Answer: Predictive analytics for early detection of students at risk of falling below the 75% attendance threshold.

Q48: How did you validate your research findings?

Answer: Through empirical testing of the system latency, cost-tracking of AI tokens (zero), and functional verification of policy lookups.

Q49: Is your code modular? Why?

Answer: Yes, I separated logic into `app.py`, `chatbot.py`, and `database.py` to ensure high maintainability and easier unit testing.

Q50: Why should a student use your bot instead of Google?

Answer: Because the bot provides verified, institutional 'single-source-of-truth' policies that are specific to Chandigarh University and not general info.